

лекция 7. Централизованное резервное копирование и восстановление

Backup нужен не ради факта создания копии, а ради гарантированного восстановления сервиса

Учебный сценарий: web01, vpn01 и docker01 передают копии на backup01

Основной инструмент лаборатории: BorgBackup
rsync рассматривается как транспорт и mirror, Restic — как альтернативный инструмент

Главный результат: студент должен уметь объяснить, проверить и восстановить резервную копию

Цель лекции

- Понять отличие backup от RAID, snapshot и replication
- Уметь объяснить RPO, RTO и правило 3-2-1
- Уметь описать архитектуру backup-сервера и ограничения доступа
- Уметь объяснить базовые команды rsync, BorgBackup и Restic
- Уметь обосновать, почему test restore важнее успешного exit code

Учебные вопросы

1. RAID, snapshot, replication и backup
2. Полное, дифференциальное и инкрементное копирование; правило 3-2-1; RPO и RTO
3. Backup-сервер, сервисная учетная запись и SSH-ключ
4. rsync, BorgBackup и Restic: инкременты, шифрование и дедупликация
5. Исключения, retention и удаление старых копий
6. Контроль успешности, целостность и тестовое восстановление

Учебный сценарий лекции

web01 содержит веб-конфигурации и каталог /srv/web-data

vpn01 содержит OpenVPN-конфигурации и PKI-материалы без публикации секретов в Git

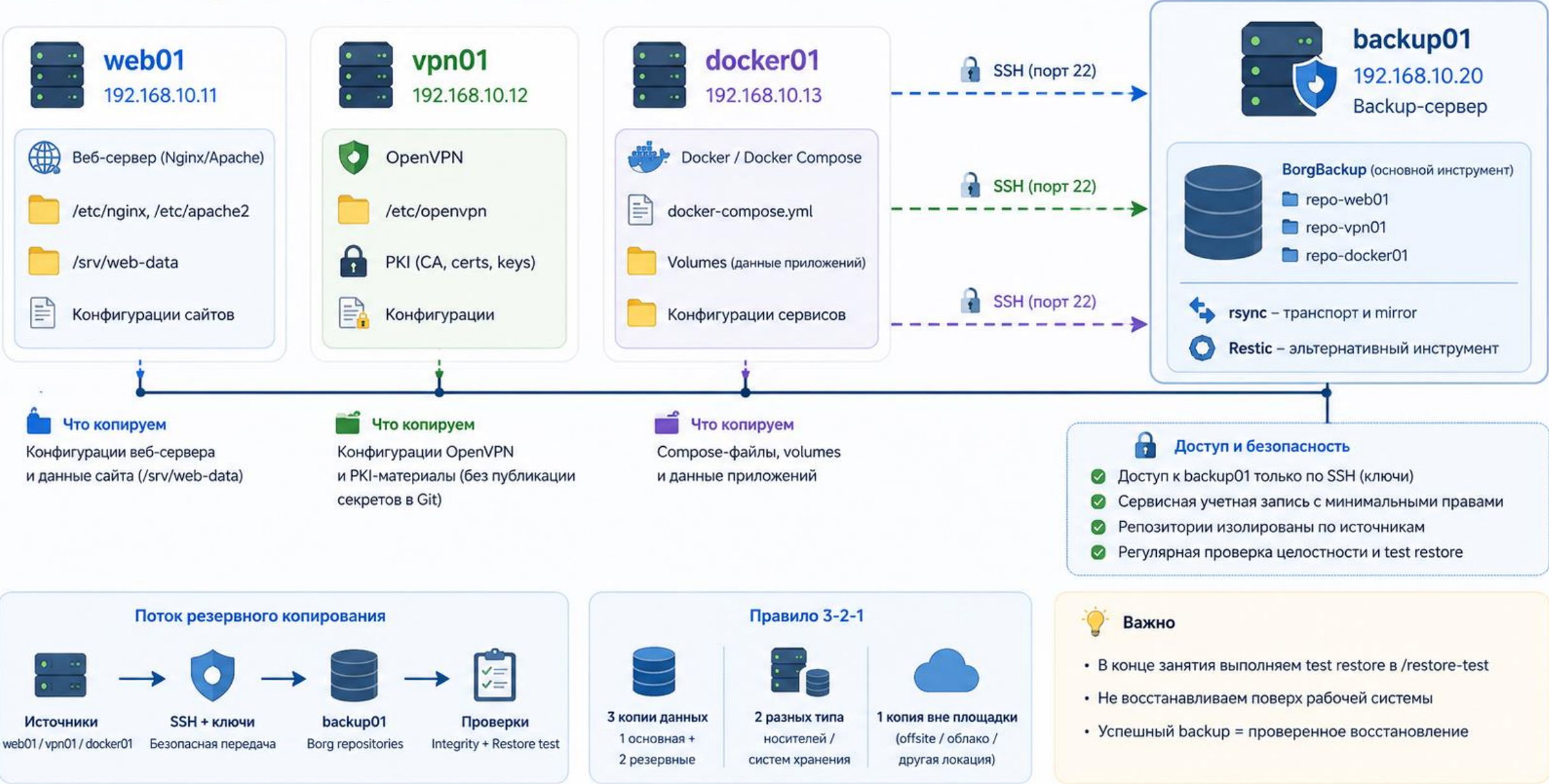
docker01 содержит Compose-файлы, volumes и данные приложений

backup01 принимает копии по SSH и хранит отдельные repository по источникам

В конце занятия проверяется восстановление в /restore-test, а не поверх рабочей системы

Топология: web01, vpn01, docker01 и backup01

Централизованное резервное копирование с backup-сервером



1. RAID, snapshot, replication и backup

RAID повышает доступность дисковой подсистемы, но не защищает от удаления файла пользователем

Snapshot фиксирует состояние тома или виртуальной машины на момент времени

Replication переносит изменения на другой узел или площадку

Backup создает восстановительную точку, которую можно проверить и использовать для restore

Ошибка новичка: считать любую вторую копию полноценным backup

Схема: RAID, snapshot, replication и backup

Каждый механизм решает свою задачу. Полная защита достигается их совместным использованием.

1. RAID

Доступность дисковой подсистемы

RAID 1 / RAID 5 / RAID 10



Данные распределяются между несколькими дисками в одном массиве.

Что обеспечивает

- ✓ Работу при отказе одного (или нескольких) дисков
- ✓ Высокую скорость чтения/записи
- ✓ Повышенную доступность данных

На что не защищает

- ✗ Удаление/изменение файлов пользователем
- ✗ Шифрование данных вредоносным ПО
- ✗ Сбои ОС, ошибки конфигурации, вирусы
- ✗ Пожар, кража, физическое уничтожение сервера

2. Snapshot

Зафиксированное состояние во времени

Снимки состояния



Фиксирует состояние тома/файловой системы на определённый момент.

Что обеспечивает

- ✓ Быстрый откат к предыдущему состоянию
- ✓ Удобно перед обновлениями и изменениями
- ✓ Минимальное влияние на производительность (на большинстве СХД)

На что не защищает

- ✗ Отказ хранилища, массива или сервера
- ✗ Удаление snapshot'ов злоумышленником
- ✗ Длительное хранение (обычно ограничено)
- ✗ Защиту от вымогателей без изоляции

3. Replication

Копия на другом узле или площадке

Основной сайт



Репликация
(синхронная/
асинхронная)

Второй сайт



Передаёт изменения на другой сервер или в другой дата-центр.

Что обеспечивает

- ✓ Отказоустойчивость и быстрое переключение
- ✓ Работа сервиса при падении площадки
- ✓ Географическое распределение данных

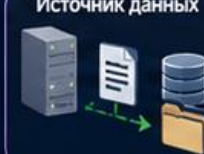
На что не защищает

- ✗ Мгновенное копирование ошибок и удалений
- ✗ Вирусы и шифровальщики (копируются тоже)
- ✗ Долгосрочное хранение версий
- ✗ Случайные изменения конфигурации

4. Backup

Восстановительная точка

Источник данных



Backup
(по расписанию)

Backup-хранилище



Создаёт копии данных, которые можно проверить и восстановить на другом узле или на том же.

Что обеспечивает

- ✓ Восстановление после любых логических сбоев
- ✓ История версий и долгосрочное хранение
- ✓ Возможность восстановления на «чистом» сервере
- ✓ Изоляцию (при правильной настройке)

На что не защищает

- ✗ Если копии доступны на запись злоумышленнику
- ✗ Если хранится на том же отказавшем ресурсе
- ✗ Без проверки копии могут оказаться «мусором»



Главный вывод

RAID, snapshot и replication повышают доступность и устойчивость, но только backup даёт настоящие восстановительные точки. Используйте их вместе по правилу 3-2-1.



RAID

Доступность

+



Snapshot

Быстрый откат

+



Replication

Отказоустойчивость

+



Backup

Восстановление

Правило 3-2-1



3

3 копии данных
1 основная + 2 резервные



2

2 разных типа носителей
например: диск + лента/облако



1

1 копия вне площадки
офсайт или облако

RAID не защищает от логических ошибок

RAID1 сохранит доступность при отказе одного диска

Если администратор удалил каталог, удаление синхронно отразится на массиве

Если файл зашифрован вредоносным ПО, RAID сохранит уже зашифрованную версию

RAID не дает истории версий и не отвечает на вопрос: каким файл был вчера

RAID — это доступность, а не восстановительная точка

Snapshot и replication решают другие задачи

Snapshot удобен для быстрого отката перед обновлением или рискованным изменением

Snapshot часто зависит от того же хранилища или гипервизора

Replication полезна для отказоустойчивости и быстрого переключения

Replication может мгновенно перенести ошибку или шифрование на вторую площадку

Backup должен быть изолирован от части рисков основного сервера

Васкир защищает только от изолированных рисков

Если исходный сервер имеет постоянный доступ на удаление архивов, атакующий может уничтожить копии

Если васкир хранится на том же RAID-массиве, отказ массива уничтожит и рабочие данные, и копии

Если васкир использует те же учетные данные администратора, компрометация расширяет ущерб

Для защиты нужны отдельные credentials, минимальные права и независимая retention policy

Для серьезной защиты применяют offline или immutable copy

2. Типы копирования, правило 3-2-1, RPO и RTO

Full backup содержит полный набор выбранных данных

Incremental backup хранит изменения относительно предыдущей точки

Differential backup хранит изменения относительно последней полной копии

Для BorgBackup и Restic классические термины применяются условно

Современные инструменты часто используют дедупликацию и хранилище блоков

Полная копия проста, но дорогая по объему

Full backup проще всего понять и восстановить

Минус: каждый запуск повторно хранит большой объем данных

Подходит для небольших каталогов и начальной точки истории

В больших системах полная копия часто комбинируется с инкрементами

Для тестового стенда полная копия удобна как учебный первый шаг

Инкремент и дифференциал экономят место

Incremental backup быстрее и компактнее, но восстановление зависит от цепочки точек

Differential backup проще восстановить, но размер растёт между полными копиями

Дедуплицирующие инструменты не всегда показывают студенту классическую цепочку full плюс incremental

Важно не название типа копии, а возможность восстановить нужное состояние

После каждого изменения политики нужно делать test restore

Правило 3-2-1 требует независимости копий

3 экземпляра: рабочие данные плюс две резервные копии

2 типа хранения: например локальное хранилище и удаленное хранилище

1 копия вне площадки или изолирована от основного отказа

Три backup-файла рядом на одном диске не выполняют смысл правила

Копии должны отличаться по рискам, правам доступа и месту хранения

Современное уточнение 3-2-1-1-0

3 экземпляра данных

2 типа хранения

1 копия вне площадки

1 копия offline или immutable

0 ошибок по результатам проверки восстановления

RPO показывает допустимую потерю данных

RPO — Recovery Point Objective, допустимый возраст восстановительной точки

Если RPO равен 1 час, бизнес допускает потерю данных не более чем за 1 час

Расписание backup само по себе не гарантирует RPO

Если три последних задания завершились ошибкой, фактический RPO уже больше трех часов

Мониторинг должен считать возраст последнего подтвержденного успешного backup

RTO показывает время восстановления сервиса

RTO — Recovery Time Objective, допустимое время простоя сервиса

В RTO входит обнаружение аварии, решение о восстановлении и подготовка сервера

Также входят получение backup, восстановление прав, запуск службы и проверка приложения

Иногда нужно переключить DNS, IP-адрес или клиентов

Команда `borg extract` за 5 минут еще не означает RTO 5 минут

3. Backup-сервер, сервисная учетная запись и SSH-ключ

Backup-сервер — отдельная роль инфраструктуры для хранения и проверки резервных копий

Он должен иметь достаточное место, журналирование, мониторинг и ограниченные права клиентов

Для каждого источника желательно выделять отдельный repository или каталог

Доступ к backup01 должен быть более строгим, чем доступ к обычному учебному серверу

Компрометация backup01 может затронуть несколько серверов сразу

Нужно выбрать модель Push или Pull

Push: исходный сервер сам отправляет копию на backup01

Pull: backup01 подключается к исходному серверу и забирает данные

В лаборатории выбираем Push-модель как более простую для первого стенда

При этом права на backup01 должны запрещать доступ к чужим repository и произвольное удаление

В реальной системе модель выбирается по рискам и требованиям безопасности

Push-модель проще, но требует защиты repository

Преимущество: web01 сам читает свои локальные файлы и отправляет копию

Преимущество: backup01 не получает SSH-доступ ко всем исходным серверам

Недостаток: компрометация web01 может угрожать его backup repository

Нужно ограничить backup credential только нужной командой и каталогом

Нужно отделять права web01, vpn01 и docker01 на backup01

Pull-модель централизует расписание, но расширяет доверие

Преимущество: расписанием управляет backup01

Преимущество: исходный сервер может дать только ограниченный read access

Недостаток: компрометация backup01 открывает путь к нескольким источникам

Pull-модель часто удобна для централизованного администрирования

В базовой лаборатории ее достаточно рассмотреть как архитектурную альтернативу

Service Account снижает ущерб от ошибки

Service Account — отдельная учетная запись для автоматической операции

Для backup не используют личную учетную запись администратора

Для каждого источника лучше создать отдельного пользователя: backup-web01, backup-vpn01, backup-docker01

Пользователь web01-backup не должен читать или менять /backups/vpn01

Единый пользователь backup допустим в учебной лаборатории только как упрощение

SSH-ключ для backup нужно ограничивать

Просто добавить public key в `authorized_keys` недостаточно

Ключ можно ограничить source IP, запретом PTY и запретом forwarding

Ключ можно привязать к фиксированной команде приема backup

Пример концепции: `from='192.168.30.10',no-pty,no-agent-forwarding,command='/usr/local/sbin/backup-receive-web01'`

В файле `authorized_keys` такая запись должна находиться одной строкой

Приватный ключ без passphrase — компромисс автоматизации

Автоматическое задание backup не может вводить пароль интерактивно

Ключ без passphrase допустим только при строгом ограничении прав и назначения

Права каталога: `chmod 700 ~/.ssh`

Права приватного ключа: `chmod 600 ~/.ssh/id_ed25519`

Права `authorized_keys`: `chmod 600
~/.ssh/authorized_keys`

SSH Host Key защищает от подмены сервера

Ключ пользователя подтверждает клиента, а

known_hosts подтверждает сервер

Не следует бездумно использовать

StrictHostKeyChecking=no

Команда: `ssh-keyscan -H backup01 >>`

`~/.ssh/known_hosts`

Назначение: добавить host key backup01 в

`known_hosts`

Важно: fingerprint нужно сверить через доверенный

канал, потому что `ssh-keyscan` сам по себе не

доказывает подлинность

sudoers для backup должен вызывать wrapper, а не всё подряд

Опасная строка: backup ALL=(ALL) NOPASSWD: ALL

Даже разрешение /usr/bin/rsync может быть слишком широким

Безопаснее разрешить root-owned wrapper script с фиксированными путями

Пример: backup ALL=(root) NOPASSWD:
/usr/local/sbin/backup-export-web01

Wrapper не должен принимать произвольные аргументы от пользователя backup

4. rsync, BorgBackup и Restic

rsync эффективно синхронизирует файлы между каталогами и серверами

BorgBackup создает зашифрованный дедуплицирующий repository с архивами

Restic выполняет похожую задачу и поддерживает разные backend-хранилища

Для лаборатории основным инструментом выбран BorgBackup

rsync используется для сравнения mirror-подхода и подготовки безопасных примеров

rsync нужно сначала запускать в dry-run

Первый учебный источник: /srv/backup-source/, а не весь /etc

Команда: `sudo rsync -aHAXn /srv/backup-source/ backup@backup01:/backups/web01/current/`

Ключ -n означает dry-run: показать действия без копирования

После проверки убрать -n: `sudo rsync -aHAX /srv/backup-source/`

`backup@backup01:/backups/web01/current/`

Для первого примера не используем `--delete`

Завершающий slash в rsync меняет результат

Команда rsync -а /source/ destination/ копирует содержимое каталога source

Команда rsync -а /source destination/ создает destination/source/

Для backup scripts это критично, потому что меняется структура восстановления

Ошибка в одном slash может особенно сильно навредить вместе с --delete

Перед автоматизацией нужно проверить результат в тестовом каталоге

Ключи -aHAX и --numeric-ids имеют условия

- a означает archive mode: рекурсивное копирование с базовыми атрибутами
- H сохраняет hard links
- A сохраняет POSIX ACL, если есть поддержка и права
- X сохраняет extended attributes, если они поддерживаются
- numeric-ids помогает сохранить UID/GID как числа, а не интерпретировать их по именам

Обычный rsync — это mirror, а не версионный backup

Если источник удалил файл, следующий rsync с --delete удалит файл и на приемнике

Mirror удобен для быстрого получения текущего состояния

Mirror плохо защищает от поздно замеченного удаления или шифрования

Версионность можно строить через snapshot, --link-dest, Borg, Restic или backup-систему

--delete добавляют только после dry-run и понимания последствий

BorgBackup в лекции фиксируется как основной инструмент

Borg создает repository и архивы внутри него

Borg поддерживает дедупликацию: одинаковые блоки не хранятся повторно

Borg поддерживает шифрование repository

Команды зависят от версии Borg, поэтому в лаборатории нужно зафиксировать установленную версию

Примеры ниже ориентированы на классический синтаксис Borg 1.x

Инициализация Borg repository

Переменная: export

BORG_REPO='ssh://backup@backup01/./backups/borg/web01'

Переменная: export

BORG_PASSPHRASE='учебная_фраза'

Команда: borg init --encryption=repokey-blake2

Назначение: создать зашифрованный repository Borg

Важно: passphrase нельзя хранить в Git или прямо в рабочем сценарии

Создание архива Borg

Команда: `borg create '::{hostname}-{now:%Y-%m-%d_%H-%M-%S}' /srv/backup-source`

Назначение: создать архив с именем, включающим host и точное время

Время в имени предотвращает конфликт при повторном запуске в тот же день

Проверка списка: `borg list`

Информация о repository: `borg info`

Ключ Borg и passphrase нужно сохранить отдельно

При `repokey` ключ шифрования хранится в `repository` в зашифрованном виде

Для восстановления нужны `repository`, `passphrase`, доступ и совместимый Borg

Если `passphrase` потерян, `backup` становится недоступен

Пример экспорта ключа: `borg key export`

`"$BORG_REPO" /secure-offline/web01-borg-key`

Экспортированный ключ и `passphrase` нельзя хранить только рядом с `repository`

Borg restore выполняется в отдельный каталог

Нельзя бездумно выполнять `borg extract` из корня файловой системы

Команда: `mkdir -p /restore-test/web01`

Команда: `cd /restore-test/web01`

Команда: `borg extract '::archive-name'`

После извлечения проверяют файлы, права, синтаксис конфигурации и только потом переносят данные

Restic можно рассмотреть как альтернативу

Переменная: export

RESTIC_REPOSITORY='sftp:backup@backup01:/backups/restic/web01'

Переменная: export

RESTIC_PASSWORD_FILE='/root/.config/restic/password'

Команда: restic init

Команда: restic backup /srv/backup-source

Команды проверки: restic snapshots, restic restore latest --target /restore-test/web01, restic check

Консистентность приложений важнее копирования файлов

Копирование файлов работающей СУБД через rsync может дать непригодный набор данных

Для баз данных используют logical dump, physical backup tool или application quiesce

Примеры концепций: mysqldump, pg_dump, pg_basebackup

Для Docker нельзя просто копировать /var/lib/docker во время работы daemon

Нужно определить Compose-файлы, named volumes, database dump, user data и процедуру quiesce

Перед backup нужно проверить, что источник смонтирован

Если отдельный диск не смонтировался, каталог /srv/data может быть пустым на root filesystem

Backup успешно скопирует пустой каталог и завершится кодом 0

Команда: `mountpoint -q /srv/data`

Назначение: проверить, что точка монтирования действительно активна

Команда: `findmnt -n -o SOURCE /srv/data`

5. Исключения, retention и удаление старых копий

Исключения уменьшают объем backup, но могут случайно убрать важные данные

Retention policy определяет, сколько восстановительных точек хранить

Prune удаляет старые точки по политике

Integrity check проверяет читаемость и структуру repository

Эти операции нужно планировать, журналировать и тестировать

Исключения должны быть проверены восстановлением

Пример исключения: `--exclude /var/cache/`

Паттерны зависят от инструмента и точки source

После настройки нужно проверить, что мусор
исключен, а важные файлы остались

Файл исключений лучше хранить отдельно:

`/etc/backup/web01.exclude`

Файл исключений можно версионировать, если он не
содержит секретов

Не все журналы являются мусором

Большие logs можно исключить, если они уже централизованно отправляются в syslog или log platform

Но журналы могут понадобиться для расследования инцидента

Журналы могут требоваться для аудита и правил хранения

Решение об исключении принимается по политике, а не только по размеру

В backup включают данные, необходимые для восстановления и выполнения требований хранения

Retention для Borg и Restic задается командами

Пример политики: 7 daily, 4 weekly, 6 monthly

Borg: `borg prune --list --keep-daily=7 --keep-weekly=4 --keep-monthly=6 "$BORG_REPO"`

Restic: `restic forget --keep-daily 7 --keep-weekly 4 --keep-monthly 6 --prune`

Перед удалением полезен `dry run` или просмотр списка

Нельзя запускать два `prune` одновременно

Prune и check могут быть тяжелыми операциями

Полный integrity check может занять много времени и создать нагрузку

Нужен частый легкий контроль и периодический полный check

Нужно контролировать свободное место и окно обслуживания

Нельзя совмещать backup и prune без понимания блокировок repository

После обслуживания нужно сохранить журнал результата

Расписание лучше показывать через systemd timer

Cron прост, но systemd timer лучше интегрируется с journald и зависимостями

Фрагмент timer: OnCalendar=daily

Фрагмент timer: Persistent=true

Фрагмент timer: RandomizedDelaySec=10m

Для service полезны Type=oneshot, Nice=10 и
IOSchedulingClass=best-effort

Нужна защита от параллельного запуска

Если старое задание не завершилось, новый запуск может создать нагрузку или конфликт с prune

Команда: `flock -n /run/lock/web01-backup.lock
/usr/local/sbin/web01-backup.sh`

Назначение: не запускать второй экземпляр backup job

Borg и Restic имеют собственные блокировки repository

Организационно параллельные задания все равно нужно контролировать

6. Контроль успешности, целостность и тестовое восстановление

Exit code 0 не доказывает полноту backup

Integrity check проверяет repository, но не доказывает запуск приложения

Test restore показывает, можно ли реально извлечь данные и поднять сервис

Backup без проверки восстановления — это предположение, а не гарантия

RPO и RTO подтверждаются только измерениями и журналами успешных задач

Exit code 0 может скрывать плохую копию

Задание может успешно скопировать пустой каталог

Задание может создать архив объемом 2 КБ вместо обычных 20 ГБ

Источник мог быть не смонтирован

Копия могла не содержать нужных файлов приложения

Поэтому мониторинг должен проверять объем, возраст, список архивов и ожидаемые пути

Monitoring backup должен быть связан с RPO

Backup age считают от последней подтвержденной успешной копии

Если RPO равен 24 часа, а последняя успешная копия 30 часов назад, состояние критическое

Нужно учитывать нормальную длительность задания и допустимую задержку

Отдельно контролируется задание, которое выполняется слишком долго

Мониторинг должен замечать массовое удаление архивов и аномально малый размер

Repository Integrity и Application Restore — разные проверки

Integrity check показывает, что repository читается и его структуры не повреждены

Test restore показывает, что данные реально извлекаются

Application restore показывает, что сервис запускается и работает

Checksum не доказывает, что были скопированы нужные данные

Для приемки backup нужны оба уровня: техническая целостность и прикладное восстановление

Test Restore должен быть неразрушающим

Базовый вариант: восстановление в /restore-test/web01/

Затем сравнить файлы с эталоном или ожидаемым набором

Проверить владельцев, права и важные конфигурационные файлы

Выполнить syntax check: например `sudo nginx -t` или `sudo sshd -t`

Разрушительный тест выполнять только при наличии snapshot или понятной процедуры возврата

Восстановление /etc не означает восстановление сервера

Для восстановления сервиса нужны пакеты нужных версий

Нужны системные пользователи, группы, права и владельцы

Нужны конфигурации, данные, сертификаты, ключи и secrets

Могут понадобиться firewall rules, DNS-записи, SELinux или AppArmor policy

Инструкция должна называться по сервису: Restore Nginx Service, а не просто Extract /etc

Backup Server нужно защищать от заполнения

Команда: `df -h` — проверить свободное место

Команда: `df -i` — проверить inode

Команда: `borg info` — посмотреть размер и статистику Borg repository

Один источник не должен заполнить все хранилище backup01

Практика: отдельные каталоги, quotas или filesystem для web01, vpn01 и docker01

Диаграмма: цикл backup, проверка и restore

Надёжность достигается не созданием копии, а регулярной проверкой и реальным восстановлением



Лабораторный цикл должен пройти весь путь восстановления

Создать service account и SSH key

Проверить host key backup01 и ограничения
authorized_keys

Инициализировать Borg repository с шифрованием

Создать минимум две резервные копии после
изменения тестового файла

Настроить retention, выполнить check и восстановить
архив в /restore-test

Критерии приемки лабораторной работы

Есть журнал последнего успешного backup

Backup age не нарушает заданный RPO

Repository проходит выбранную проверку целостности

Restore выполнен в отдельный каталог без перезаписи

live-системы

Студент может объяснить, какие данные не попали в backup и почему

Измерено фактическое время восстановления сервиса

Итоги лекции

Backup отличается от RAID, snapshot и replication тем, что должен давать проверяемую точку восстановления

Модель Push или Pull нужно выбирать осознанно и ограничивать права доступа

rsync полезен для mirror и транспортировки, но BorgBackup лучше подходит для версионного backup в лаборатории

RPO связан с последней подтвержденной успешной копией, а RTO — со всем временем восстановления сервиса

Контрольные вопросы

Почему RAID и replication не заменяют backup?

Чем Push-модель отличается от Pull-модели?

Почему rsync с --delete нельзя показывать без dry-run?

Что означает RPO и почему оно не равно расписанию backup?

Почему Borg extract нужно выполнять в отдельный каталог?

Какие проверки кроме exit code нужны для контроля backup?